CNSCC.405, Lab Session 1-Part 1

Objectives: The main aim of this session is to learn:

1) How to start Matlab?
2) The modular structure of Matlab and problems that can be solved
3) Learning how to write a function in Matlab
4) Basic programing including *for loops* and *while loops* structures.

1. Starting Matlab:

To start Matlab, double click on the Matlab icon on the desktop. The main Matlab window with the command line prompt ">>" should appear.

Matlab is an interpreted language. This means that the instructions you give to it are processed and interpreted by the computer directly, as opposed to for instance C: in C, when you write a program, you first need to compile it before it can be executed by the machine. This means that at any moment, all the variables are in the memory of the computer (equivalent to debug mode of C/C++) and can be readily accessed. This is a tremendous help to quickly debug programs.

There is two ways to give instructions to Matlab:

1- On the command line, after the prompt ">>": if you type an instruction, Matlab executes it and gives you the result.
2- Through a script or a function, whose code is written in a text file with the .m extension. Scripts contain lists of instructions, just as could be written on the command line. To execute a script, just type the name of the .m file (without the .m extension) on the command line1. When a script is launched, all the instructions it contains are executed successively. Functions are modular scripts that can take in arguments. They won't be detailed here, but you can look at the on-line help to find out about them.

2. Knowing where you are:

Use the pwd command to know where you are
Use the dir command to list the files in your current directory

3. Creating a vector or a matrix:

As mentioned before, matrices (and vector) as the basic element of a Matlab program. Matlab does see matrices as other languages see numbers and can perform operations in matrices directly without for loops, which comes handy for image processing...

Type :

```
>> x = [1 2 3 4 5 4 3 2 1];
>> x
>> who
>> whos
>> y = [6; 7; 8; 9; 0; 9; 8; 7; 6]
```

(note that is there is a semicolon at the end of the instruction, Matlab doesn't expand the result of the instruction)

```
>>y'
>> z = [1 2 3; 4 5 6; 7 8 9; 0 1 2]
```

[1] Matlab needs to see this file. For this, it should either be physically in your current directory (type ">>pwd" to find out where you are), or the path to its location should be known by matlab (type ">>path" to see the full path; type ">>help path" if you need to change it). Go to the File menu and Set Path sub-menu to change the path.

4. Operators: (MOST OF THEM WORK ON MATRICES DIRECTLY!)

The colon operator:

A way to define quickly some vectors is to use the ":" operator. Type for instance (and observe the results):

```
>> a = 0:10
>> b = -5:10
>> c = 0:2:10
>> d = 10:-2:-5
>> e = 0:0.01:4.2
>> f = -pi:0.01:pi
```

Type:

```
>> g = zeros(2,4)
>> h = ones(5,3)
```

Arithmetic operator:

These are the standard operators. (+ plus, - minus, / divide, * multiply, ^ exponent)

Type

```
>> 3^2
>> y'
>> x+y'
>> x*y
>> 3*x
```

The operators (/ , * and ^) have got a matrix counterpart (respectively ./ , .* and .^), which apply the operation element by element. For instance, compare: x*y and x.*y.

Type:

```
>> x^2
>> x.^2
>> x.*x
>> x*x
```

Visualisation

Type:

```
>> t =-pi:0.01:pi;
>> c = cos(t);
>> s = sin(t);
```

Plotting functions:

Type:

```
>> figure
>> plot(s)
>> figure
>> plot(t,s)
>> figure
>> plot(t,s,'g',t,c,'r')
```

Type help plot if you haven't already done it to see more possibilities.
Modify the axis, put a title and labels on the x- and y- axes by using:

xlabel('x axis'), ylabel('x axis')

Add a title, a label on the x-axis.

5. Writing an m-File Function

The following example shows how to write an M-file for the following function.

$$f(x_1, x_2) = x_1^2 - 2x_1x_2 + 6x_1 + x_2^2 - 6x_2$$

The M-file that computes this function must accept a row vector x of length 2, corresponding to the variables x_1 and x_2, and return a scalar equal to the value of the function at x. To write the M-file, do the following steps:

1. Select **New** in the MATLAB **File** menu.
2. Select **M-File**. This opens a new M-file in the editor.
3. In the M-file, enter the following two lines of code:

```
function z = my_fun(x)
z = x(1)^2 - 2*x(1)*x(2) + 6*x(1) + x(2)^2 - 6*x(2);
```

4. Save the M-file in your current directory.

To check that the M-file returns the correct value, enter

```
>> my_fun([2 3])

ans =

   -5
```

6. Getting Help

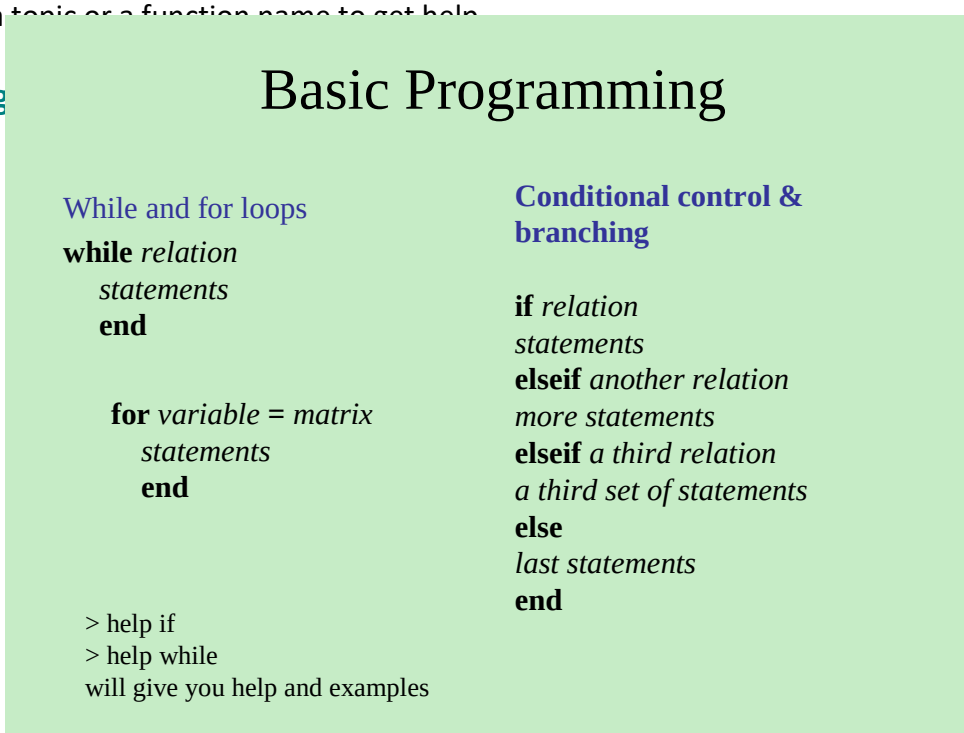
To get help on a particular function, type ***help functionname***. For example,

```
>> help zeros

ZEROS  Zeros array.
   ZEROS(N) is an N-by-N matrix of zeros.

   ZEROS(M,N) or ZEROS([M,N]) is an M-by-N matrix of zeros.

   ZEROS(M,N,P,...) or ZEROS([M N P ...]) is an M-by-N-by-P-by-... array of
   zeros.

   ZEROS(SIZE(A)) is the same size as A and all zeros.

   ZEROS with no arguments is the scalar 0.

   ZEROS(M,N,...,CLASSNAME) or ZEROS([M,N,...],CLASSNAME) is an
   M-by-N-by-... array of zeros of class CLASSNAME.

   Example:
     x = zeros(2,3,'int8');

   See also eye, ones.

   Reference page in Help browser
```

```
doc zeros
```

Or you can open the Help Browser by choosing Help/Matlab Help from the menu and then search for a topic or a function name to get help

7. Basic Prog



7.1. For Loops

Syntax of the **for** loop is shown below

```
for k = array
        commands
end
```

The commands between the for and end statements are executed for all values stored in the **array**.

Suppose that one-need values of the sine function at eleven evenly spaced points **pi*n/10**, for **n = 0, 1, …, 10**. To generate the numbers in question one can use the **for** loop

```
for n=0:10
        x(n+1) = sin(pi*n/10);
end
x

x =
Columns 1 through 7
0 0.3090 0.5878 0.8090 0.9511 1.0000 0.9511
Columns 8 through 11
0.8090 0.5878 0.3090 0.0000
```

The **for** loops can be nested

```
H = zeros(5);
```

```
for k=1:5
        for l=1:5
                H(k,l) = 1/(k+l-1);
        end
end
H

H =
1.0000 0.5000 0.3333 0.2500 0.2000
0.5000 0.3333 0.2500 0.2000 0.1667
0.3333 0.2500 0.2000 0.1667 0.1429
0.2500 0.2000 0.1667 0.1429 0.1250
0.2000667 0.1429 0.1250 0.1111
```

7.2. While Loop

Syntax of the **while** loop is

```
while expression
        statements
end
```

This loop is used when the programmer does not know the number of repetitions a priori.

Here is an almost trivial problem that requires a use of this loop. Suppose that the number pi is divided by 2. The resulting quotient is divided by 2 again. This process is continued till the current quotient is less than or equal to 0.01. What is the largest quotient that is greater than 0.01? To answer this question we write a few lines of code

```
q = pi;
while q > 0.01
        q = q/2;
end
q

q =
0.0061
```

7.3. If-Else Conditional Statements

IF Conditionally execute statements. The general form of the **IF** statement is

```
IF expression
  statements
ELSEIF expression
  statements
ELSE
  statements
END
```

The statements are executed if the conditional expression has all true or non-zero elements. The ELSE and ELSEIF parts are optional. More ELSEIF parts can be used as well as nested IF's.

The expression is usually of the form *expr rop expr* where *rop* is ==, <, >, <=, >=, or ~=.

Example:

```
Function pass = isQualified(attendance, grade_average)

if ((attendance >= 0.90) && (grade_average >= 60))
  pass = 1;
else
  pass = 0;
end;

>> isQualified(0.95, 55)

ans =

    0
```